

Отчёт по лабораторной работе №2 2026

Компьютерный практикум по статистическому анализу данных

Лисенков Егор Романович

Содержание

1 Выполнение лабораторной работы

2 Цель работы

Изучение нескольких структур данных, реализованных в языке Julia: кортежей, словарей, множеств и массивов. Освоение операций создания, индексации, преобразования и комбинирования этих структур для решения прикладных задач. Получение практических навыков работы в интерактивной среде Jupyter Lab с ядром Julia и оформления результатов вычислений в виде блокнота.

3 Задание

Используя Jupyter Lab, повторить примеры раздела 2.2 методических указаний: продемонстрировать работу общих для всех структур функций `isempty`, `length`, `in`, `unique`, `reduce`, `maximum` и `minimum`, а также операции над кортежами, словарями, множествами и массивами. Выполнить задания для самостоятельной работы раздела 2.4: операции над множествами, создание массивов заданной структуры различными способами, преобразования векторов, статистический анализ случайных выборок, работу с пакетом `Primes` и вычисление сумм. Оформить отчёт с пояснениями, скриншотами и листингами.

4 Конкретные задачи

Создать каталог `labs/lab02` в репозитории курса и запустить Jupyter Lab с ядром Julia. Создать блокнот `lab02.ipynb` и воспроизвести в нём примеры раздела 2.2: общие функции коллекций, кортежи и именованные кортежи, словари с операциями добавления, удаления и слияния, множества с операциями объединения, пересечения и разности, массивы различной размерности и операции над ними. Выполнить задания самостоятельной

работы: вычисление множества P через пересечения и объединения, операции над множествами разнотипных элементов, построение массивов заданной структуры, статистический анализ случайных выборок длины 250, генерацию простых чисел с помощью пакета Primes и вычисление трёх сумм. Зафиксировать результаты в системе контроля версий Git.

5 Теоретическое введение

В языке Julia реализовано несколько базовых структур данных, над которыми определён общий набор функций: `isempty` проверяет, пуста ли структура; `length` возвращает число элементов; `in` проверяет принадлежность элемента структуре; `unique` возвращает коллекцию уникальных элементов; `reduce` свёртывает структуру согласно заданному бинарному оператору; `maximum` и `minimum` возвращают наибольший и наименьший элементы соответственно.

Кортеж (Tuple) — неизменяемая индексируемая последовательность элементов произвольных типов, индексация начинается с единицы. Кортеж задаётся перечислением элементов в круглых скобках, возможен именованный кортеж, к полям которого можно обращаться по имени через точку. Поскольку кортеж неизменяем, он удобен для хранения фиксированных наборов значений и возврата нескольких результатов из функции.

Словарь (Dict) — неупорядоченный набор пар «ключ — значение», задаваемый синтаксисом `Dict{key => value}`. Основные операции: `keys` и `values` для получения коллекций ключей и значений, `pairs` для получения пар, `haskey` для проверки наличия ключа, добавление элемента через присваивание по новому ключу, удаление пары через `pop!`, слияние словарей через `merge`, при котором в случае конфликта ключей побеждает значение из словаря, переданного последним аргументом.

Множество (Set) — неупорядоченная совокупность уникальных элементов, соответствующая математическому понятию множества. Множество создаётся из итерируемого объекта, при этом дубликаты отбрасываются, а строка трактуется как набор символов. Поддерживаются операции `union` (объединение), `intersect` (пересечение), `setdiff` (разность), `issubset` (проверка вхождения одного множества в другое), `issetequal` (проверка эквивалентности), а также добавление и удаление элементов через `push!` и `pop!`.

Массив — коллекция упорядоченных элементов, размещённая в многомерной сетке; векторы и матрицы являются частными случаями массивов. Массивы создаются перечислением элементов в квадратных скобках (запятая разделяет строки, пробел — столбцы), генераторами вида `[f(i) for i in range if condition]`, функциями `rand`, `ones`, `zeros`, `fill`, `repeat`, `collect`.

Основные операции: `length`, `ndims`, `size`, `copy`, `reshape` для изменения формы, `transpose` и штрих для транспонирования, `sort` для сортировки по заданному измерению. Индексация поддерживает срезы через двоеточие, диапазоны, списки индексов, а также логические маски; функция `findall` возвращает индексы элементов, удовлетворяющих условию, в виде `CartesianIndex`.

6 Выполнение лабораторной работы

Работа началась с перехода в каталог репозитория курса `komputerni-praktikum-2026-2027` и входа в каталог лабораторной работы `labs/lab02`, предварительно созданный для хранения файлов второй лабораторной. Из этого каталога была выполнена команда `jupyter lab`, запустившая сервер Jupyter Lab. В терминале появились служебные сообщения `ServerApp` об импорте пакетов `jupyterlab` и `jupyter_lsp`, а также предупреждение о том, что функция `_jupyter_server_extension_points` не найдена в `jupyter_lsp` и вместо неё использована `_jupyter_server_extension_paths`; это предупреждение является штатным для данной версии расширения и не влияет на работу среды. Запуск сервера из каталога `labs/lab02` гарантирует, что создаваемый блокнот будет сохранён именно в каталоге лабораторной работы и впоследствии попадёт в систему контроля версий (рис. 1).

```
egorlisenkov@fedora: ~$ cd ~/komputerni-praktikum-2026-2027
egorlisenkov@fedora: ~/komputerni-praktikum-2026-2027$ cd labs/lab02
egorlisenkov@fedora: ~/komputerni-praktikum-2026-2027/labs/lab02$ jupyter lab
[I 2026-09-26 14:15:56.980 ServerApp] Package jupyterlab took 0.0000s to import
[I 2026-09-26 14:15:57.048 ServerApp] Package jupyter_lsp took 0.0654s to import
[W 2026-09-26 14:15:57.048 ServerApp] A `_jupyter_server_extension_points' function was not
found in jupyter_lsp. Instead, a `_jupyter_server_extension_paths' function was found and
```

Рис. 1: Запуск Jupyter Lab из каталога лабораторной работы

В открывшейся среде Jupyter Lab был создан блокнот `lab02.ipynb` с ядром Julia 1.10. Первая ячейка с текстом заголовка «Лабораторная работа 2. Структуры данных» и строкой «Выполнил: Лисенков Е.Р.» была по ошибке выполнена в режиме `Code`, что привело к ошибке `UndefVarError: Выполнил not defined`, поскольку вторая строка не являлась комментарием и интерпретатор попытался использовать неизвестное имя как переменную. После перевода ячейки в режим `Markdown` заголовок отобразился корректно, что наглядно продемонстрировало различие режимов ячейки. Далее в ячейке режима `Code` были проверены общие функции коллекций на примере вектора `v = [1, 2, 3, 3]`: `isempty` вернула `false`, `length` вернула 4, проверка `in(2, v)` дала `true`, а `0 in v` — `false`, `unique` вернула `[1, 2, 3]`, `reduce(+, v)` дала сумму 9, `maximum` и `minimum` вернули 3 и 1 соответственно (рис. 2).

```

# Definieren pattern 2. Strukturplan
Baukasten: Baukasten.FP

UnknuffError: Baukasten not defined

Stacktrace:
[1] top-level scope
 @ Includ.jl

v = [1, 2, 3, 3]
println(lumpFP ., lumpFP{ell})
println(lumpFP ., lumpFP{ell})
println("v: ", [v[i], v[i] + 1] * 8 in v)
println("vmax: ", v[4])
println("vmin: ", v[1])
println("maxima: ", maxima(v))
println("minima: ", minima(v))

```

Рис. 2: Создание блокнота, ошибка режима Code и общие функции коллекций

Следующим шагом были воспроизведены примеры с кортежами и словарями. Созданы пустой кортеж `t_empty`, кортеж строк `favoritelang = ("Python", "Julia", "R")`, целочисленный кортеж `x1 = (1, 2, 3)`, кортеж разнотипных элементов `x2 = (1, 2.0, "tmp")` и именованный кортеж `x3 = (a = 2, b = 1 + 2)`. Проверены длина `length(x2) = 3`, доступ по индексам `x2[1]`, `x2[2]`, `x2[3]`, арифметическая операция `c = x1[2] + x1[3] = 5`, доступ к полям именованного кортежа `x3.a = 2`, `x3.b = 3` и по индексу `x3[2] = 3`, а также проверка вхождения `in("tmp", x2) = true` и `0 in x2 = false`. Затем создан словарь `phonebook` с ключами «Иванов И.И.» (значение — кортеж из двух номеров) и «Бухгалтерия»; выведены коллекции ключей, значений и пар, проверено наличие ключа через `haskey (true)`. В словарь добавлена пара «Сидоров П.С.» => "555-3344", после чего пара «Иванов И.И.» удалена через `pop!`. Слияние словарей `a` и `b`, имеющих общий ключ `"bar"`, продемонстрировало, что в `merge(a, b)` значение ключа `"bar"` берётся из `b (13.0)`, а в `merge(b, a)` — из `a (42.0)` (рис. 3).

```

1 # sample / maximum: 3 / 3
2
3 x = 10
4
5 f = lambda x: "Python", "C++", "C#"
6
7 a = 12, 21, 31
8
9 b = 12, 21, 31, "foo"
10
11 c = 9, 10, 12
12
13 print("Python" in f, "C++" in f, "C#" in f)
14
15 print("Python" in a, "C++" in a, "C#" in a, "12" in a, "21", "31")
16
17 print("Python" in b, "C++" in b, "C#" in b, "12" in b, "21", "31", "foo")
18
19 print("Python" in c, "C++" in c, "C#" in c, "9" in c, "10", "12")
20
21
22 length(a) = 3
23
24 minmax(a) = 12, 21, 31
25
26 minmax(b) = 12, 21, 31, "foo"
27
28 minmax(c) = 9, 10, 12
29
30 minmax(d) = 9, 10, 12, "foo"
31
32 minmax(e) = 9, 10, 12, "foo"
33
34
35 phonebook = {
36     "name": "John",
37     "number": "0151-5306-1087",
38     "email": "john@python.com"
39 }
40
41 print(phonebook["name"])
42 print(phonebook["number"])
43 print(phonebook["email"])
44
45 print(phonebook.get("name", "None"))
46 print(phonebook.get("number", "None"))
47 print(phonebook.get("email", "None"))
48
49
50 phonebook["number"] = "0151-5306-1084"
51
52 print(phonebook)
53
54 del phonebook["email"]
55
56 print(phonebook)
57
58
59 # dict.items()
60
61 a = {
62     "name": "John",
63     "number": "0151-5306-1087",
64     "email": "john@python.com"
65 }
66
67 b = dict(a.items())
68
69 print(b)
70
71
72 # dict.keys()
73
74 a = {
75     "name": "John",
76     "number": "0151-5306-1087",
77     "email": "john@python.com"
78 }
79
80 b = dict(a.keys())
81
82 print(b)
83
84
85 # dict.values()
86
87 a = {
88     "name": "John",
89     "number": "0151-5306-1087",
90     "email": "john@python.com"
91 }
92
93 b = dict(a.values())
94
95 print(b)
96
97
98 # dict.get()
99
100 a = {
101     "name": "John",
102     "number": "0151-5306-1087",
103     "email": "john@python.com"
104 }
105
106 b = dict(a.get("name", "None"))
107
108 print(b)
109
110
111 # dict.setdefault()
112
113 a = {
114     "name": "John",
115     "number": "0151-5306-1087",
116     "email": "john@python.com"
117 }
118
119 b = dict(a.setdefault("name", "None"))
120
121 print(b)
122
123
124 # dict.pop()
125
126 a = {
127     "name": "John",
128     "number": "0151-5306-1087",
129     "email": "john@python.com"
130 }
131
132 b = dict(a.pop("name", "None"))
133
134 print(b)
135
136
137 # dict.popitem()
138
139 a = {
140     "name": "John",
141     "number": "0151-5306-1087",
142     "email": "john@python.com"
143 }
144
145 b = dict(a.popitem())
146
147 print(b)
148
149
150 # dict.clear()
151
152 a = {
153     "name": "John",
154     "number": "0151-5306-1087",
155     "email": "john@python.com"
156 }
157
158 b = dict(a.clear())
159
160 print(b)
161
162
163 # dict.copy()
164
165 a = {
166     "name": "John",
167     "number": "0151-5306-1087",
168     "email": "john@python.com"
169 }
170
171 b = dict(a.copy())
172
173 print(b)
174
175
176 # dict.fromkeys()
177
178 a = {
179     "name": "John",
180     "number": "0151-5306-1087",
181     "email": "john@python.com"
182 }
183
184 b = dict.fromkeys(a)
185
186 print(b)
187
188
189 # dict.update()
190
191 a = {
192     "name": "John",
193     "number": "0151-5306-1087",
194     "email": "john@python.com"
195 }
196
197 b = dict(a.update({"name": "John", "number": "0151-5306-1087", "email": "john@python.com"}))
198
199 print(b)
200
201
202 # dict.setdefault()
203
204 a = {
205     "name": "John",
206     "number": "0151-5306-1087",
207     "email": "john@python.com"
208 }
209
210 b = dict(a.setdefault("name", "None"))
211
212 print(b)
213
214
215 # dict.setdefault()
216
217 a = {
218     "name": "John",
219     "number": "0151-5306-1087",
220     "email": "john@python.com"
221 }
222
223 b = dict(a.setdefault("name", "None"))
224
225 print(b)
226
227
228 # dict.setdefault()
229
230 a = {
231     "name": "John",
232     "number": "0151-5306-1087",
233     "email": "john@python.com"
234 }
235
236 b = dict(a.setdefault("name", "None"))
237
238 print(b)
239
240
241 # dict.setdefault()
242
243 a = {
244     "name": "John",
245     "number": "0151-5306-1087",
246     "email": "john@python.com"
247 }
248
249 b = dict(a.setdefault("name", "None"))
250
251 print(b)
252
253
254 # dict.setdefault()
255
256 a = {
257     "name": "John",
258     "number": "0151-5306-1087",
259     "email": "john@python.com"
260 }
261
262 b = dict(a.setdefault("name", "None"))
263
264 print(b)
265
266
267 # dict.setdefault()
268
269 a = {
270     "name": "John",
271     "number": "0151-5306-1087",
272     "email": "john@python.com"
273 }
274
275 b = dict(a.setdefault("name", "None"))
276
277 print(b)
278
279
280 # dict.setdefault()
281
282 a = {
283     "name": "John",
284     "number": "0151-5306-1087",
285     "email": "john@python.com"
286 }
287
288 b = dict(a.setdefault("name", "None"))
289
290 print(b)
291
292
293 # dict.setdefault()
294
295 a = {
296     "name": "John",
297     "number": "0151-5306-1087",
298     "email": "john@python.com"
299 }
300
301 b = dict(a.setdefault("name", "None"))
302
303 print(b)
304
305
306 # dict.setdefault()
307
308 a = {
309     "name": "John",
310     "number": "0151-5306-1087",
311     "email": "john@python.com"
312 }
313
314 b = dict(a.setdefault("name", "None"))
315
316 print(b)
317
318
319 # dict.setdefault()
320
321 a = {
322     "name": "John",
323     "number": "0151-5306-1087",
324     "email": "john@python.com"
325 }
326
327 b = dict(a.setdefault("name", "None"))
328
329 print(b)
330
331
332 # dict.setdefault()
333
334 a = {
335     "name": "John",
336     "number": "0151-5306-1087",
337     "email": "john@python.com"
338 }
339
340 b = dict(a.setdefault("name", "None"))
341
342 print(b)
343
344
345 # dict.setdefault()
346
347 a = {
348     "name": "John",
349     "number": "0151-5306-1087",
350     "email": "john@python.com"
351 }
352
353 b = dict(a.setdefault("name", "None"))
354
355 print(b)
356
357
358 # dict.setdefault()
359
360 a = {
361     "name": "John",
362     "number": "0151-5306-1087",
363     "email": "john@python.com"
364 }
365
366 b = dict(a.setdefault("name", "None"))
367
368 print(b)
369
370
371 # dict.setdefault()
372
373 a = {
374     "name": "John",
375     "number": "0151-5306-1087",
376     "email": "john@python.com"
377 }
378
379 b = dict(a.setdefault("name", "None"))
380
381 print(b)
382
383
384 # dict.setdefault()
385
386 a = {
387     "name": "John",
388     "number": "0151-5306-1087",
389     "email": "john@python.com"
390 }
391
392 b = dict(a.setdefault("name", "None"))
393
394 print(b)
395
396
397 # dict.setdefault()
398
399 a = {
400     "name": "John",
401     "number": "0151-5306-1087",
402     "email": "john@python.com"
403 }
404
405 b = dict(a.setdefault("name", "None"))
406
407 print(b)
408
409
410 # dict.setdefault()
411
412 a = {
413     "name": "John",
414     "number": "0151-5306-1087",
415     "email": "john@python.com"
416 }
417
418 b = dict(a.setdefault("name", "None"))
419
420 print(b)
421
422
423 # dict.setdefault()
424
425 a = {
426     "name": "John",
427     "number": "0151-5306-1087",
428     "email": "john@python.com"
429 }
430
431 b = dict(a.setdefault("name", "None"))
432
433 print(b)
434
435
436 # dict.setdefault()
437
438 a = {
439     "name": "John",
440     "number": "0151-5306-1087",
441     "email": "john@python.com"
442 }
443
444 b = dict(a.setdefault("name", "None"))
445
446 print(b)
447
448
449 # dict.setdefault()
450
451 a = {
452     "name": "John",
453     "number": "0151-5306-1087",
454     "email": "john@python.com"
455 }
456
457 b = dict(a.setdefault("name", "None"))
458
459 print(b)
460
461
462 # dict.setdefault()
463
464 a = {
465     "name": "John",
466     "number": "0151-5306-1087",
467     "email": "john@python.com"
468 }
469
470 b = dict(a.setdefault("name", "None"))
471
472 print(b)
473
474
```

Рис. 3: Операции над кортежами и словарями

Далее были воспроизведены примеры с множествами и создание массивов. Созданы множество целых чисел $A = \text{Set}([1, 3, 4, 5])$ и множество символов $B = \text{Set}(\text{"abracadabra"})$, которое после удаления дубликатов содержит пять уникальных символов. Проверена эквивалентность множеств: `issetequal(S1, S2)` для непересекающихся множеств дала `false`, а `issetequal(S3, S4)` для множеств с одинаковым составом элементов, но разными дубликатами, дала `true`. Выполнены объединение `union(S1, S2) = Set([4, 2, 3, 1])`, пересечение `intersect(S1, S3) = Set([2, 1])`, разность `setdiff(S3, S1) = Set([3])` и проверка `issubset(S1, S4) = true`. Продемонстрированы добавление элемента `push!(S4, 99)` и удаление `pop!(S4)`. Затем созданы массивы: пустые массивы с абстрактным и конкретными типами `Int64` и `Float64`, вектор-столбец `a = [1, 2, 3]`, вектор-строка `b = [1 2 3]`, матрица `Amat 3x3`, случайные массивы `c_rand` размером `1x8`, `C_rand` размером `2x3` и трёхмерный `D_rand` размером `4x3x2`, а также массивы-

Завершающим этапом основной части стали операции над массивами и демонстрация индексации. Для матрицы `Aop` 3x3 вычислены `length = 9` (общее число элементов), `ndims = 2` (число размерностей), `size = (3, 3)` и `size(Aop, 1) = 3`. Созданы массивы из единиц `ones(5)` и `ones(2, 3)`, массив нулей `zeros(4)`, массив `fill(3.5, (3, 2))` и продемонстрирована функция `repeat` для вектора и матрицы. Одномерный массив `collect(1:12)` преобразован через `reshape` в матрицу 2x6, затем транспонирован двумя способами: штрихом `b'` и функцией `transpose(b)`, оба дали матрицу 6x2. Для случайной матрицы `ar` размером 10x5 целых чисел из диапазона `[10, 20]`, сгенерированной с фиксированным зерном `Random.seed!(42)`, выполнены срезы: весь столбец 2, столбцы 2 и 5, столбцы 2:4, строки 2, 4, 6 со столбцами 1 и 5, строка 1 от столбца 3 до конца. Выполнены сортировки по столбцам `sort(ar, dims=1)` и по строкам `sort(ar, dims=2)`, построена логическая маска `ar .> 14` и получены индексы удовлетворяющих условию элементов через `findall` в виде списка `CartesianIndex` (рис. 5).

Рис. 5: Операции над массивами, индексация и сортировка

объединение попарных пересечений исходных множеств; присутствующий в формулировке повторяющийся член $A \cap B$ не изменяет результат благодаря идемпотентности операции объединения. Итоговое множество $P = \text{Set}([0, 4, 7, 9, 3, 1])$ содержит шесть элементов: 0, 1, 3, 4, 7 и 9. В задании 2 продемонстрированы операции над множествами, составленными из элементов разных типов: множество M1 объединяет целое число 1, вещественное 2.5, строку “Julia” и символ ‘c’, а множество M2 содержит вещественное число, строку и кортеж (1, 2). Объединение $\text{union}(M1, M2)$ дало $\text{Set}(\text{Any}[(1, 2), 'c', \text{“Julia”}, 2.5, 1])$, пересечение $\text{intersect}(M1, M2) = \text{Set}(\text{Any}[\text{“Julia”}, 2.5])$, разность $\text{setdiff}(M1, M2) = \text{Set}(\text{Any}['c', 1])$, а проверка $\text{issubset}(\text{intersect}(M1, M2), M1)$ вернула true, подтвердив, что пересечение является подмножеством первого множества (рис. 6).

```

A = Set{Int, Float64, String, Char}
B = Set{Int, Float64, String}
C = Set{Int, Float64, String, Char, Tuple{Int, Int}}
P = union(intersect(A, B), intersect(A, C), intersect(B, C))
println("P = ", P)

M1 = Set{Int, Float64, String, Char}
M2 = Set{Float64, String, Tuple{Int, Int}}
println("union: ", union(M1, M2))
println("intersect: ", intersect(M1, M2))
println("setdiff: ", setdiff(M1, M2))
println("issubset: ", issubset(intersect(M1, M2), M1))

union: Set{Any{Int, Float64, String, Char, Tuple{Int, Int}}}
intersect: Set{Any{String, Float64}}
setdiff: Set{Any{Char, Int}}
issubset: true

```

Рис. 6: Задания 1 и 2: операции над множествами

В задании 3 различными способами были созданы массивы заданной структуры при $N = 25$, что удовлетворяет требованию N больше 20. Массив $\text{arr_31} = \text{collect}(1:N)$ содержит последовательность от 1 до 25; массив $\text{arr_32} = \text{collect}(N:-1:1)$ — убывающую последовательность от 25 до 1; массив arr_33 получен вертикальной конкатенацией $[\text{collect}(1:N); \text{collect}(N:-1:1)]$ и содержит 49 элементов, образуя последовательность с возрастанием до 25 и последующим убыванием до 1. Далее задан массив $\text{tmp} = [4, 6, 3]$; массив $\text{arr_35} = \text{fill}(\text{tmp}[1], 10)$ содержит десять четвёрок; $\text{arr_36} = \text{repeat}(\text{tmp}, 10)$ повторяет всю тройку десять раз (30 элементов); arr_37 собран конкатенацией $\text{fill}(\text{tmp}[1], 11)$, $\text{fill}(\text{tmp}[2], 10)$ и $\text{fill}(\text{tmp}[3], 10)$, что дало 31 элемент; arr_38 составлен из блоков repeat длиной 10, 20 и 30 элементов, его длина равна 60. В пункте 3.9 сформирован массив $\text{arr_39} = [2\text{tmp}[1], 2\text{tmp}[2], \text{fill}(2*\text{tmp}[3], 4)\dots] = [8, 12, 6, 6, 6, 6]$, и с помощью функции $\text{count}(==(6), \text{arr_39})$ подсчитано число вхождений шестёрки: результат равен 4, что и выведено на экран (рис. 7).

```

N = 25
arr_31 = collect(1:N)
arr_32 = collect(N:-1:1)
arr_33 = [collect(1:N); collect(N:-1:1)]
println(arr_31); println(arr_32); println(arr_33)

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25]
[25, 24, 23, 22, 21, 20, 19, 18, 17, 16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 24, 23, 22, 21, 20, 19, 18, 17, 16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]

tmp = [4, 6, 3]
arr_35 = fill(tmp[1], 10)
arr_36 = repeat(tmp, 10)
arr_37 = [fill(tmp[1], 11); fill(tmp[2], 10); fill(tmp[3], 10)]
arr_38 = [repeat(tmp[1], 10); repeat(tmp[2], 20); repeat(tmp[3], 30)]
println(arr_35); println(arr_36); println(arr_37); println("длина arr_38 = ", length(arr_38))

[4, 4, 4, 4, 4, 4, 4, 4, 4, 4]
[4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3]
[4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3]
длина arr_38 = 60

arr_39 = [2*tmp[1], 2*tmp[2], fill(2*tmp[3], 4)...]
count6 = count(==(6), arr_39)
println("массив: ", arr_39)
println("число 6 встречается раз: ", count6)

массив: [8, 12, 6, 6, 6, 6]
число 6 встречается раз: 4

```

Рис. 7: Задание 3: создание массивов заданной структуры

В пунктах 3.10–3.13 продолжено построение массивов-генераторов. В пункте 3.10 на сетке $x = 3:0.1:6$ (31 точка) вычислен вектор $y = \exp.(x) .* \cos.(x)$ и найдено

его среднее значение с помощью функции mean из пакета Statistics: среднее значение $y = 53.11374594642971$. В пункте 3.11 сгенерирован вектор пар $\text{vec_311} = [(0.1i, 0.2j) \text{ for } (i, j) \text{ in zip}(3:3:36, 1:3:34)]$, содержащий двенадцать кортежей от (0.3, 0.2) до (3.6, 6.8), что демонстрирует одновременную итерацию по двум диапазонам с помощью функции zip. В пункте 3.12 построен вектор $\text{vec_312} = [2^i / i \text{ for } i \text{ in } 1:25]$, первые элементы которого равны [2.0, 2.0, 2.6667, 4.0, 6.4]. В пункте 3.13 сформирован вектор строк $\text{vec_313} = [\text{"fn"}i \text{ for } i \text{ in } 1:30]$, начинающийся с ["fn1", "fn2", "fn3", "fn4", "fn5"] и завершающийся строкой "fn30"; интерполяция переменной внутри строковой литеральной конструкции позволила получить именованные элементы без ручного перечисления (рис. 8).

```
using Statistics
x30 = 2:3:36
y34 = 1:3:34
println("генерируем y = 10.122222222222222")

vec_311 = [(0.3i, 0.2j) for (i, j) in zip(x30, y34)]
N = 25
vec_312 = 2^i / i for i in 1:N
vec_313 = ["fn"i for i in 1:30]
println(vec_311, println(vec_312), println(vec_313))
(0.3, 0.2), (0.6, 0.4), (0.9, 0.6), (1.2, 0.8), (1.5, 1.0), (1.8, 1.2), (2.1, 1.4), (2.4, 1.6), (2.7, 1.8), (3.0, 2.0), (3.3, 2.2), (3.6, 2.4), (3.9, 2.6), (4.2, 2.8), (4.5, 3.0), (4.8, 3.2), (5.1, 3.4), (5.4, 3.6), (5.7, 3.8), (6.0, 4.0), (6.3, 4.2), (6.6, 4.4), (6.9, 4.6), (7.2, 4.8), (7.5, 5.0), (7.8, 5.2), (8.1, 5.4), (8.4, 5.6), (8.7, 5.8), (9.0, 6.0), (9.3, 6.2), (9.6, 6.4), (9.9, 6.6), (10.2, 6.8)
```

Рис. 8: Задания 3.10–3.13: векторы-генераторы и статистика

Наиболее объёмным стало задание 3.14, в котором сформированы целочисленные векторы x и y длины $n = 250$ как случайные выборки из совокупности от 0 до 999 с фиксированным зерном $\text{Random.seed!}(123)$. Построены векторы $v_1 = y[2:\text{end}] - x[1:\text{end}-1]$ длины 249, $v_2 = x[1:\text{end}-2] + 2 \cdot x[2:\text{end}-1] - x[3:\text{end}]$ длины 248 и $v_3 = \sin.(y[1:\text{end}-1]) ./ \cos.(x[2:\text{end}])$ длины 249, а также вычислена сумма $\text{sum}(\exp.(-x[2:\text{end}]) ./ (x[1:\text{end}-1] + 10)) = 0.00017065210565897441$. С помощью логической маски $\text{mask} = y .> 600$ выбраны элементы y , превосходящие 600, определены их индексы через $\text{findall}(\text{mask})$ и соответствующие элементы x на тех же индексных позициях. Сформирован вектор $v_4 = \text{sqrt}(\text{abs}.(x - \text{mean}(x)))$. Статистический анализ показал: 57 элементов y отстоят от максимума не более чем на 200; в x содержится 129 чётных и 121 нечётный элемент; 30 элементов x кратны 7; первые десять элементов x , отсортированных по возрастанию y , равны [471, 799, 152, 34, 100, 404, 681, 597, 311, 84]; десятка наибольших элементов x (top-10) равна [996, 993, 993, 986, 983, 974, 959, 953, 949, 943]; число уникальных элементов x составило 227 (рис. 9).

```
using Random, Statistics
Random.seed!(123)
n = 250
x = rand{Int8, n}
y = rand{Int8, n}

v_1 = y[2:end] - x[1:end-1]
v_2 = x[1:end-2] + 2 * x[2:end-1] - x[3:end]
v_3 = sin.(y[1:end-1]) ./ cos.(x[2:end])
sum_exp = sum(exp.(-x[2:end]) ./ (x[1:end-1] + 10))
println("sum_exp = ", sum_exp)

mask = y .> 600
indices_y = findall(mask)
indices_x = findall(mask)

v_4 = sqrt(abs(x - mean(x)))
println("v_4 = ", v_4)

println("статистический анализ")
println("max(x) = ", maximum(x))
println("min(x) = ", minimum(x))
println("max(y) = ", maximum(y))
println("min(y) = ", minimum(y))
println("sum(x) = ", sum(x))
println("sum(y) = ", sum(y))
println("unique(x) = ", unique(x))
println("unique(y) = ", unique(y))
println("count{<=200}(y) = ", count{<=200}(y))
println("count{<=200}(x) = ", count{<=200}(x))
println("count{<=200}(v_1) = ", count{<=200}(v_1))
println("count{<=200}(v_2) = ", count{<=200}(v_2))
println("count{<=200}(v_3) = ", count{<=200}(v_3))
println("count{<=200}(v_4) = ", count{<=200}(v_4))
println("count{<=200}(v_5) = ", count{<=200}(v_5))
println("count{<=200}(v_6) = ", count{<=200}(v_6))
println("count{<=200}(v_7) = ", count{<=200}(v_7))
println("count{<=200}(v_8) = ", count{<=200}(v_8))
println("count{<=200}(v_9) = ", count{<=200}(v_9))
println("count{<=200}(v_10) = ", count{<=200}(v_10))
println("count{<=200}(v_11) = ", count{<=200}(v_11))
println("count{<=200}(v_12) = ", count{<=200}(v_12))
println("count{<=200}(v_13) = ", count{<=200}(v_13))
println("count{<=200}(v_14) = ", count{<=200}(v_14))
println("count{<=200}(v_15) = ", count{<=200}(v_15))
println("count{<=200}(v_16) = ", count{<=200}(v_16))
println("count{<=200}(v_17) = ", count{<=200}(v_17))
println("count{<=200}(v_18) = ", count{<=200}(v_18))
println("count{<=200}(v_19) = ", count{<=200}(v_19))
println("count{<=200}(v_20) = ", count{<=200}(v_20))
println("count{<=200}(v_21) = ", count{<=200}(v_21))
println("count{<=200}(v_22) = ", count{<=200}(v_22))
println("count{<=200}(v_23) = ", count{<=200}(v_23))
println("count{<=200}(v_24) = ", count{<=200}(v_24))
println("count{<=200}(v_25) = ", count{<=200}(v_25))
println("count{<=200}(v_26) = ", count{<=200}(v_26))
println("count{<=200}(v_27) = ", count{<=200}(v_27))
println("count{<=200}(v_28) = ", count{<=200}(v_28))
println("count{<=200}(v_29) = ", count{<=200}(v_29))
println("count{<=200}(v_30) = ", count{<=200}(v_30))
println("count{<=200}(v_31) = ", count{<=200}(v_31))
println("count{<=200}(v_32) = ", count{<=200}(v_32))
println("count{<=200}(v_33) = ", count{<=200}(v_33))
println("count{<=200}(v_34) = ", count{<=200}(v_34))
println("count{<=200}(v_35) = ", count{<=200}(v_35))
println("count{<=200}(v_36) = ", count{<=200}(v_36))
println("count{<=200}(v_37) = ", count{<=200}(v_37))
println("count{<=200}(v_38) = ", count{<=200}(v_38))
println("count{<=200}(v_39) = ", count{<=200}(v_39))
println("count{<=200}(v_40) = ", count{<=200}(v_40))
println("count{<=200}(v_41) = ", count{<=200}(v_41))
println("count{<=200}(v_42) = ", count{<=200}(v_42))
println("count{<=200}(v_43) = ", count{<=200}(v_43))
println("count{<=200}(v_44) = ", count{<=200}(v_44))
println("count{<=200}(v_45) = ", count{<=200}(v_45))
println("count{<=200}(v_46) = ", count{<=200}(v_46))
println("count{<=200}(v_47) = ", count{<=200}(v_47))
println("count{<=200}(v_48) = ", count{<=200}(v_48))
println("count{<=200}(v_49) = ", count{<=200}(v_49))
println("count{<=200}(v_50) = ", count{<=200}(v_50))
println("count{<=200}(v_51) = ", count{<=200}(v_51))
println("count{<=200}(v_52) = ", count{<=200}(v_52))
println("count{<=200}(v_53) = ", count{<=200}(v_53))
println("count{<=200}(v_54) = ", count{<=200}(v_54))
println("count{<=200}(v_55) = ", count{<=200}(v_55))
println("count{<=200}(v_56) = ", count{<=200}(v_56))
println("count{<=200}(v_57) = ", count{<=200}(v_57))
println("count{<=200}(v_58) = ", count{<=200}(v_58))
println("count{<=200}(v_59) = ", count{<=200}(v_59))
println("count{<=200}(v_60) = ", count{<=200}(v_60))
println("count{<=200}(v_61) = ", count{<=200}(v_61))
println("count{<=200}(v_62) = ", count{<=200}(v_62))
println("count{<=200}(v_63) = ", count{<=200}(v_63))
println("count{<=200}(v_64) = ", count{<=200}(v_64))
println("count{<=200}(v_65) = ", count{<=200}(v_65))
println("count{<=200}(v_66) = ", count{<=200}(v_66))
println("count{<=200}(v_67) = ", count{<=200}(v_67))
println("count{<=200}(v_68) = ", count{<=200}(v_68))
println("count{<=200}(v_69) = ", count{<=200}(v_69))
println("count{<=200}(v_70) = ", count{<=200}(v_70))
println("count{<=200}(v_71) = ", count{<=200}(v_71))
println("count{<=200}(v_72) = ", count{<=200}(v_72))
println("count{<=200}(v_73) = ", count{<=200}(v_73))
println("count{<=200}(v_74) = ", count{<=200}(v_74))
println("count{<=200}(v_75) = ", count{<=200}(v_75))
println("count{<=200}(v_76) = ", count{<=200}(v_76))
println("count{<=200}(v_77) = ", count{<=200}(v_77))
println("count{<=200}(v_78) = ", count{<=200}(v_78))
println("count{<=200}(v_79) = ", count{<=200}(v_79))
println("count{<=200}(v_80) = ", count{<=200}(v_80))
println("count{<=200}(v_81) = ", count{<=200}(v_81))
println("count{<=200}(v_82) = ", count{<=200}(v_82))
println("count{<=200}(v_83) = ", count{<=200}(v_83))
println("count{<=200}(v_84) = ", count{<=200}(v_84))
println("count{<=200}(v_85) = ", count{<=200}(v_85))
println("count{<=200}(v_86) = ", count{<=200}(v_86))
println("count{<=200}(v_87) = ", count{<=200}(v_87))
println("count{<=200}(v_88) = ", count{<=200}(v_88))
println("count{<=200}(v_89) = ", count{<=200}(v_89))
println("count{<=200}(v_90) = ", count{<=200}(v_90))
println("count{<=200}(v_91) = ", count{<=200}(v_91))
println("count{<=200}(v_92) = ", count{<=200}(v_92))
println("count{<=200}(v_93) = ", count{<=200}(v_93))
println("count{<=200}(v_94) = ", count{<=200}(v_94))
println("count{<=200}(v_95) = ", count{<=200}(v_95))
println("count{<=200}(v_96) = ", count{<=200}(v_96))
println("count{<=200}(v_97) = ", count{<=200}(v_97))
println("count{<=200}(v_98) = ", count{<=200}(v_98))
println("count{<=200}(v_99) = ", count{<=200}(v_99))
println("count{<=200}(v_100) = ", count{<=200}(v_100))
println("count{<=200}(v_101) = ", count{<=200}(v_101))
println("count{<=200}(v_102) = ", count{<=200}(v_102))
println("count{<=200}(v_103) = ", count{<=200}(v_103))
println("count{<=200}(v_104) = ", count{<=200}(v_104))
println("count{<=200}(v_105) = ", count{<=200}(v_105))
println("count{<=200}(v_106) = ", count{<=200}(v_106))
println("count{<=200}(v_107) = ", count{<=200}(v_107))
println("count{<=200}(v_108) = ", count{<=200}(v_108))
println("count{<=200}(v_109) = ", count{<=200}(v_109))
println("count{<=200}(v_110) = ", count{<=200}(v_110))
println("count{<=200}(v_111) = ", count{<=200}(v_111))
println("count{<=200}(v_112) = ", count{<=200}(v_112))
println("count{<=200}(v_113) = ", count{<=200}(v_113))
println("count{<=200}(v_114) = ", count{<=200}(v_114))
println("count{<=200}(v_115) = ", count{<=200}(v_115))
println("count{<=200}(v_116) = ", count{<=200}(v_116))
println("count{<=200}(v_117) = ", count{<=200}(v_117))
println("count{<=200}(v_118) = ", count{<=200}(v_118))
println("count{<=200}(v_119) = ", count{<=200}(v_119))
println("count{<=200}(v_120) = ", count{<=200}(v_120))
println("count{<=200}(v_121) = ", count{<=200}(v_121))
println("count{<=200}(v_122) = ", count{<=200}(v_122))
println("count{<=200}(v_123) = ", count{<=200}(v_123))
println("count{<=200}(v_124) = ", count{<=200}(v_124))
println("count{<=200}(v_125) = ", count{<=200}(v_125))
println("count{<=200}(v_126) = ", count{<=200}(v_126))
println("count{<=200}(v_127) = ", count{<=200}(v_127))
println("count{<=200}(v_128) = ", count{<=200}(v_128))
println("count{<=200}(v_129) = ", count{<=200}(v_129))
println("count{<=200}(v_130) = ", count{<=200}(v_130))
println("count{<=200}(v_131) = ", count{<=200}(v_131))
println("count{<=200}(v_132) = ", count{<=200}(v_132))
println("count{<=200}(v_133) = ", count{<=200}(v_133))
println("count{<=200}(v_134) = ", count{<=200}(v_134))
println("count{<=200}(v_135) = ", count{<=200}(v_135))
println("count{<=200}(v_136) = ", count{<=200}(v_136))
println("count{<=200}(v_137) = ", count{<=200}(v_137))
println("count{<=200}(v_138) = ", count{<=200}(v_138))
println("count{<=200}(v_139) = ", count{<=200}(v_139))
println("count{<=200}(v_140) = ", count{<=200}(v_140))
println("count{<=200}(v_141) = ", count{<=200}(v_141))
println("count{<=200}(v_142) = ", count{<=200}(v_142))
println("count{<=200}(v_143) = ", count{<=200}(v_143))
println("count{<=200}(v_144) = ", count{<=200}(v_144))
println("count{<=200}(v_145) = ", count{<=200}(v_145))
println("count{<=200}(v_146) = ", count{<=200}(v_146))
println("count{<=200}(v_147) = ", count{<=200}(v_147))
println("count{<=200}(v_148) = ", count{<=200}(v_148))
println("count{<=200}(v_149) = ", count{<=200}(v_149))
println("count{<=200}(v_150) = ", count{<=200}(v_150))
println("count{<=200}(v_151) = ", count{<=200}(v_151))
println("count{<=200}(v_152) = ", count{<=200}(v_152))
println("count{<=200}(v_153) = ", count{<=200}(v_153))
println("count{<=200}(v_154) = ", count{<=200}(v_154))
println("count{<=200}(v_155) = ", count{<=200}(v_155))
println("count{<=200}(v_156) = ", count{<=200}(v_156))
println("count{<=200}(v_157) = ", count{<=200}(v_157))
println("count{<=200}(v_158) = ", count{<=200}(v_158))
println("count{<=200}(v_159) = ", count{<=200}(v_159))
println("count{<=200}(v_160) = ", count{<=200}(v_160))
println("count{<=200}(v_161) = ", count{<=200}(v_161))
println("count{<=200}(v_162) = ", count{<=200}(v_162))
println("count{<=200}(v_163) = ", count{<=200}(v_163))
println("count{<=200}(v_164) = ", count{<=200}(v_164))
println("count{<=200}(v_165) = ", count{<=200}(v_165))
println("count{<=200}(v_166) = ", count{<=200}(v_166))
println("count{<=200}(v_167) = ", count{<=200}(v_167))
println("count{<=200}(v_168) = ", count{<=200}(v_168))
println("count{<=200}(v_169) = ", count{<=200}(v_169))
println("count{<=200}(v_170) = ", count{<=200}(v_170))
println("count{<=200}(v_171) = ", count{<=200}(v_171))
println("count{<=200}(v_172) = ", count{<=200}(v_172))
println("count{<=200}(v_173) = ", count{<=200}(v_173))
println("count{<=200}(v_174) = ", count{<=200}(v_174))
println("count{<=200}(v_175) = ", count{<=200}(v_175))
println("count{<=200}(v_176) = ", count{<=200}(v_176))
println("count{<=200}(v_177) = ", count{<=200}(v_177))
println("count{<=200}(v_178) = ", count{<=200}(v_178))
println("count{<=200}(v_179) = ", count{<=200}(v_179))
println("count{<=200}(v_180) = ", count{<=200}(v_180))
println("count{<=200}(v_181) = ", count{<=200}(v_181))
println("count{<=200}(v_182) = ", count{<=200}(v_182))
println("count{<=200}(v_183) = ", count{<=200}(v_183))
println("count{<=200}(v_184) = ", count{<=200}(v_184))
println("count{<=200}(v_185) = ", count{<=200}(v_185))
println("count{<=200}(v_186) = ", count{<=200}(v_186))
println("count{<=200}(v_187) = ", count{<=200}(v_187))
println("count{<=200}(v_188) = ", count{<=200}(v_188))
println("count{<=200}(v_189) = ", count{<=200}(v_189))
println("count{<=200}(v_190) = ", count{<=200}(v_190))
println("count{<=200}(v_191) = ", count{<=200}(v_191))
println("count{<=200}(v_192) = ", count{<=200}(v_192))
println("count{<=200}(v_193) = ", count{<=200}(v_193))
println("count{<=200}(v_194) = ", count{<=200}(v_194))
println("count{<=200}(v_195) = ", count{<=200}(v_195))
println("count{<=200}(v_196) = ", count{<=200}(v_196))
println("count{<=200}(v_197) = ", count{<=200}(v_197))
println("count{<=200}(v_198) = ", count{<=200}(v_198))
println("count{<=200}(v_199) = ", count{<=200}(v_199))
println("count{<=200}(v_200) = ", count{<=200}(v_200))
println("count{<=200}(v_201) = ", count{<=200}(v_201))
println("count{<=200}(v_202) = ", count{<=200}(v_202))
println("count{<=200}(v_203) = ", count{<=200}(v_203))
println("count{<=200}(v_204) = ", count{<=200}(v_204))
println("count{<=200}(v_205) = ", count{<=200}(v_205))
println("count{<=200}(v_206) = ", count{<=200}(v_206))
println("count{<=200}(v_207) = ", count{<=200}(v_207))
println("count{<=200}(v_208) = ", count{<=200}(v_208))
println("count{<=200}(v_209) = ", count{<=200}(v_209))
println("count{<=200}(v_210) = ", count{<=200}(v_210))
println("count{<=200}(v_211) = ", count{<=200}(v_211))
println("count{<=200}(v_212) = ", count{<=200}(v_212))
println("count{<=200}(v_213) = ", count{<=200}(v_213))
println("count{<=200}(v_214) = ", count{<=200}(v_214))
println("count{<=200}(v_215) = ", count{<=200}(v_215))
println("count{<=200}(v_216) = ", count{<=200}(v_216))
println("count{<=200}(v_217) = ", count{<=200}(v_217))
println("count{<=200}(v_218) = ", count{<=200}(v_218))
println("count{<=200}(v_219) = ", count{<=200}(v_219))
println("count{<=200}(v_220) = ", count{<=200}(v_220))
println("count{<=200}(v_221) = ", count{<=200}(v_221))
println("count{<=200}(v_222) = ", count{<=200}(v_222))
println("count{<=200}(v_223) = ", count{<=200}(v_223))
println("count{<=200}(v_224) = ", count{<=200}(v_224))
println("count{<=200}(v_225) = ", count{<=200}(v_225))
println("count{<=200}(v_226) = ", count{<=200}(v_226))
println("count{<=200}(v_227) = ", count{<=200}(v_227))
println("count{<=200}(v_228) = ", count{<=200}(v_228))
println("count{<=200}(v_229) = ", count{<=200}(v_229))
println("count{<=200}(v_230) = ", count{<=200}(v_230))
println("count{<=200}(v_231) = ", count{<=200}(v_231))
println("count{<=200}(v_232) = ", count{<=200}(v_232))
println("count{<=200}(v_233) = ", count{<=200}(v_233))
println("count{<=200}(v_234) = ", count{<=200}(v_234))
println("count{<=200}(v_235) = ", count{<=200}(v_235))
println("count{<=200}(v_236) = ", count{<=200}(v_236))
println("count{<=200}(v_237) = ", count{<=200}(v_237))
println("count{<=200}(v_238) = ", count{<=200}(v_238))
println("count{<=200}(v_239) = ", count{<=200}(v_239))
println("count{<=200}(v_240) = ", count{<=200}(v_240))
println("count{<=200}(v_241) = ", count{<=200}(v_241))
println("count{<=200}(v_242) = ", count{<=200}(v_242))
println("count{<=200}(v_243) = ", count{<=200}(v_243))
println("count{<=200}(v_244) = ", count{<=200}(v_244))
println("count{<=200}(v_245) = ", count{<=200}(v_245))
println("count{<=200}(v_246) = ", count{<=200}(v_246))
println("count{<=200}(v_247) = ", count{<=200}(v_247))
println("count{<=200}(v_248) = ", count{<=200}(v_248))
println("count{<=200}(v_249) = ", count{<=200}(v_249))
println("count{<=200}(v_250) = ", count{<=200}(v_250))
println("count{<=200}(v_251) = ", count{<=200}(v_251))
println("count{<=200}(v_252) = ", count{<=200}(v_252))
println("count{<=200}(v_253) = ", count{<=200}(v_253))
println("count{<=200}(v_254) = ", count{<=200}(v_254))
println("count{<=200}(v_255) = ", count{<=200}(v_255))
println("count{<=200}(v_256) = ", count{<=200}(v_256))
println("count{<=200}(v_257) = ", count{<=200}(v_257))
println("count{<=200}(v_258) = ", count{<=200}(v_258))
println("count{<=200}(v_259) = ", count{<=200}(v_259))
println("count{<=200}(v_260) = ", count{<=200}(v_260))
println("count{<=200}(v_261) = ", count{<=200}(v_261))
println("count{<=200}(v_262) = ", count{<=200}(v_262))
println("count{<=200}(v_263) = ", count{<=200}(v_263))
println("count{<=200}(v_264) = ", count{<=200}(v_264))
println("count{<=200}(v_265) = ", count{<=200}(v_265))
println("count{<=200}(v_266) = ", count{<=200}(v_266))
println("count{<=200}(v_267) = ", count{<=200}(v_267))
println("count{<=200}(v_268) = ", count{<=200}(v_268))
println("count{<=200}(v_269) = ", count{<=200}(v_269))
println("count{<=200}(v_270) = ", count{<=200}(v_270))
println("count{<=200}(v_271) = ", count{<=200}(v_271))
println("count{<=200}(v_272) = ", count{<=200}(v_272))
println("count{<=200}(v_273) = ", count{<=200}(v_273))
println("count{<=200}(v_274) = ", count{<=200}(v_274))
println("count{<=200}(v_275) = ", count{<=200}(v_275))
println("count{<=200}(v_276) = ", count{<=200}(v_276))
println("count{<=200}(v_277) = ", count{<=200}(v_277))
println("count{<=200}(v_278) = ", count{<=200}(v_278))
println("count{<=200}(v_279) = ", count{<=200}(v_279))
println("count{<=200}(v_280) = ", count{<=200}(v_280))
println("count{<=200}(v_281) = ", count{<=200}(v_281))
println("count{<=200}(v_282) = ", count{<=200}(v_282))
println("count{<=200}(v_283) = ", count{<=200}(v_283))
println("count{<=200}(v_284) = ", count{<=200}(v_284))
println("count{<=200}(v_285) = ", count{<=200}(v_2
```

На завершающем этапе самостоятельной работы выполнены задания 4 и 5 и подготовлены вычисления задания 6. В задании 4 создан массив `squares = [i^2 for i in 1:100]`, хранящий квадраты всех целых чисел от 1 до 100; первые десять элементов равны [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]. В задании 5 подключён пакет `Primes`: в выводе ячейки зафиксированы обновление реестра пакетов, разрешение версий и добавление зависимостей `Primes v0.5.7` и `IntegerMathUtils v0.1.4` в файлы `Project.toml` и `Manifest.toml` проекта, после чего сформирован массив `myprimes = primes(1000)` и подготовлены вызовы для определения количества простых чисел, 89-го наименьшего простого и среза массива. В задании 6 подготовлены ячейки вычисления трёх сумм: суммы выражения $i^3 + 4 \cdot i^2$ для i от 10 до 100, суммы выражения $2^i + 3^i / i^2$ для i от 1 до 25 и суммы ряда с отношением произведений, вычисляемой циклом `for k in 2:38` (рис. 10).

```

julia> squares = [i^2 for i in 1:100]
100-element Vector{Int64}:
 1  4  9 16 25 36 49 64 81 100 121 144 169 196 225 256 289 324 361 400 441 484 529 576 625 676 729 784 841 900 961 1024 1089 1156 1225 1296 1369 1444 1521 1600 1681 1764 1849 1936 2025 2116 2209 2304 2401 2500 2601 2704 2809 2916 3025 3136 3249 3364 3481 3600 3721 3844 3969 4096 4225 4356 4489 4624 4761 4900 5041 5184 5329 5476 5625 5776 5929 6084 6241 6400 6561 6724 6891 7060 7231 7404 7581 7760 7941 8124 8311 8500 8691 8884 9081 9280 9481 9684 9891 10100
julia> myprimes = primes(1000)
168-element Vector{Int64}:
 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97 101 103 107 109 113 127 131 137 149 151 157 163 167 173 179 181 191 193 197 199 211 223 227 229 233 239 241 251 257 263 269 271 277 281 283 293 307 311 313 317 331 337 347 349 353 359 367 373 379 383 389 397 401 409 419 421 431 433 439 443 449 457 461 463 467 479 487 491 499 503 509 521 523 541 547 557 563 569 571 577 587 593 601 607 613 617 619 626 631 637 641 643 647 653 659 661 667 671 673 677 683 687 691 697 701 709 713 727 733 739 743 751 757 761 769 773 787 797 809 811 821 823 827 829 833 839 843 853 857 859 863 869 877 881 883 887 893 899 907 911 913 917 919 923 929 937 941 947 953 959 967 971 973 977 983 989 991 997
julia> length(myprimes)
168
julia> myprimes[89]
461
julia> myprimes[90:99]
463 467 479 487 491 499 503 509 521 523
julia> prod(2:k) / prod((k+1):(2k-1))
2.1934716918435388e9
julia> sum(i^3 + 4*i^2 for i in 10:100)
26852735
julia> sum(2^i + 3^i / i^2 for i in 1:25)
2.1934716918435388e9

```

Рис. 10: Задания 4–5: квадраты целых чисел и подключение пакета `Primes`

Итоговые результаты заданий 4–6 подтверждены полным выполнением соответствующих ячеек. Массив `squares` выведен корректно. После установки пакета `Primes` получены характеристики массива простых чисел: количество простых чисел, не превосходящих 1000, равно 168, что совпадает с требуемым первым набором из 168 простых; 89-е наименьшее простое число равно 461; срез массива с 89-го по 99-й элемент включительно равен [461, 463, 467, 479, 487, 491, 499, 503, 509, 521, 523]. В задании 6 вычислены три суммы: сумма 6.1 по выражению $i^3 + 4 \cdot i^2$ для i от 10 до 100 равна 26852735; сумма 6.2 по выражению $2^i + 3^i / i^2$ для i от 1 до 25 равна 2.1934716918435388e9, причём величина определяется старшими членами слагаемого $3^i / i^2$; сумма 6.3, вычисленная циклом с отношением произведений `prod(2:k) / prod((k+1):(2k-1))`, равна 22.37818348501321. Следует отметить, что при вычислении суммы 6.3 промежуточные произведения при $k \geq 11$ превышают диапазон типа `Int64` и переполняются, поэтому полученное значение отличается от аналитической суммы сходящегося ряда (около 2.1395); этот факт наглядно иллюстрирует необходимость контроля числовых типов (переход к `Float64` или `BigInt`) при вычислениях факториального типа и стал полезным наблюдением в ходе работы (рис. 11).

Рис. 11: Итоговые результаты заданий 4–6: простые числа и вычисление сумм

В результате выполнения лабораторной работы изучены основные структуры данных языка Julia и освоены операции над ними. Практически подтверждены различия структур: неизменяемость кортежа делает его удобным для фиксированных наборов значений, словарь обеспечивает быстрый доступ по ключу и гибкое слияние, множество реализует математическую семантику объединения, пересечения и разности с автоматическим удалением дубликатов, а массив предоставляет богатый

аппарат индексации, срезов, логических масок и поэлементных операций, являясь основой численных вычислений.

Выполнение заданий самостоятельной работы показало эффективность генераторов массивов и функций `collect`, `repeat`, `fill` и `reshape` для конструирования последовательностей заданной структуры, а также удобство векторизованных операций и функций `sortperm`, `unique` и `findall` для статистического анализа случайных выборок. Подключение внешнего пакета `Primes` продемонстрировало простоту расширения функциональности Julia через встроенный менеджер пакетов. Наблюдение переполнения целочисленных произведений при вычислении факториальных сумм подчеркнуло важность осознанного выбора числовых типов при научных вычислениях.

Работа в Jupyter Lab позволила объединить исполняемый код, пояснения и результаты в едином документе, а фиксация результатов в Git обеспечила воспроизводимость и сохраняемость хода выполнения. Лабораторная работа выполнена полностью в соответствии с требованиями методического пособия: примеры раздела 2.2 воспроизведены, задания раздела 2.4 выполнены, отчёт оформлен с пояснениями, скриншотами и листингами программ.